What Do You Buy with One GPU-Hour? A Compute-Matched Study of Post-Training for Small Language Models

Anonymous submission

Abstract

Post-training methods for language models—supervised fine-tuning (SFT), rejection-sampling self-training, preference optimisation, and inference-time scaling—are usually compared under their own authors' compute budgets, which makes the reported gains hard to attribute. We instead fix the budget and ask a practitioner's question: *at the scale of a single consumer GPU, which post-training investment returns the most accuracy per GPU-hour?* We build a controlled arena over two open model sizes (0.5B and 1.5B parameters), five methods, four budget points from 5 to 60 minutes, and two seeds, and we reuse a *single* sampling pass across three of the methods so that the sampling cost is charged exactly once. [TODO: one-sentence headline finding, e.g. "We find that self-consistency decoding dominates trained post-training on mathematical reasoning at every budget below XX.X GPU-hours, while SFT remains the only method that reliably improves format compliance."] We release the harness, all adapters, and the raw per-run logs.

1 Introduction

A practitioner with one consumer GPU and a week of wall-clock time faces a concrete allocation problem: spend the time training an adapter, generating and voting over samples at inference, or collecting better data? The post-training literature is rich in methods but almost always reports each method at the budget its authors chose, on models far larger than a single 16 GB card can hold [15, 8, 12]. That literature therefore answers "does method M help?" but not "is M the best use of my next ten hours?"

This paper takes the second question seriously and makes three contributions.

1. **A compute-matched arena.** We compare SFT on gold data, rejection-sampling fine-tuning (RFT), DPO, self-consistency decoding, and an untrained prompt-only baseline under a *fixed GPU-hour budget*, sweeping four budget points. Training stops when the budget is exhausted, not when validation accuracy plateaus.

2. **A shared-sampling design.** RFT and DPO are both derived from a single pool of K samples per prompt, so the sampling cost is paid once and charged once. Without this, the arena is three times more expensive and stops being reproducible on one GPU.

3. **A format-dividend diagnostic.** We report accuracy under three answer-extraction parsers of increasing strictness, and show how much of each method's apparent gain survives the strictest one. [TODO: state the measured dividend.]

We are explicit about what this paper is not: it does not propose a new algorithm, it does not reach state of the art, and its scale is small. Its claim is scoped to the regime it measures—small open models, tens of GPU-hours—which is precisely the regime most practitioners actually occupy.

2 Related Work

Post-training pipelines. Instruction tuning and preference optimisation form the standard post-training stack [13, 15, 12], with reinforcement learning on verifiable rewards now central to reasoning models [8]; policy-gradient methods follow PPO [16]. Some pipelines now insert a *midtraining* stage between pretraining and post-training [17]. Surveys of the area [2, 3] catalogue the design space but do not attempt cross-method budget normalisation.

Does RL do more than SFT? A contested line of work asks whether RL post-training teaches new capabilities or merely redistributes what the base model already has [6, 21]. Our RFT and DPO arms are constructed from the *same* sample pool, which lets the reader separate "new data" from "new objective" effects that these studies must otherwise confound.

Self-training and rejection sampling. STaR [22] and RAFT [9] fine-tune on self-generated correct trajectories; our RFT is the sampling-based variant of this family, and it shares its sample pool with DPO by construction.

Inference-time scaling. Self-consistency [20] and compute-optimal test-time scaling [18] show that decoding-time compute can substitute for model scale. We ask the complementary question of whether it substitutes for *post-training* compute, at matched cost.

Scaling and empirical studies. [19] characterise RL post-training scaling on Qwen2.5 from 0.5B to 72B and find a predictive power law plus saturation of RL efficiency with scale; our work is a budget-normalised complement at the small end. [4] warn that high SFT scores can mislead about downstream RL behaviour, and [1] study SFT/RL interleaving. Neither normalises by GPU-hours across inference-side alternatives.

Evaluation fragility. [5] show alignment leaderboard ranks are configuration-conditional. Our three-parser reporting is a small, deliberate concession to the same concern.

3 Experimental Setup

3.1 Models and data

We use Qwen2.5-Instruct at 0.5B and 1.5B [14], in bf16 with LoRA [11] (r=16, α=32, all linear projections). Training prompts are drawn from GSM8K [7] and the algebra split of MATH [10]; evaluation uses the GSM8K test set and MATH-500. We unify the target format to "reasoning text, then `The answer is \boxed{X}.`" so that format compliance is measurable independently of correctness. A 5% dev slice of the training pool is carved out before any tuning; the test sets are never used for any decision.

3.2 Methods

Prompt-only. No training; the instruct model is queried directly.

SFT. LoRA fine-tuning on gold solutions.

RFT. We sample K=8 responses per prompt at T=0.7, keep the correct ones (verified against the gold answer), and run the same SFT recipe on them.

Method	5 m	15 m	30 m	60 m
Prompt-only	XX.X	XX.X	XX.X	XX.X
Self-consistency	XX.X	XX.X	XX.X	XX.X
SFT	XX.X	XX.X	XX.X	XX.X
RFT	XX.X	XX.X	XX.X	XX.X
DPO	XX.X	XX.X	XX.X	XX.X

Table 1: GSM8K test accuracy (standard parser, mean of 2 seeds) at fixed GPU-minute budgets, 1.5B. Generated by `src/make_tables.py`.

DPO. From the same pool we form (chosen, rejected) pairs wherever a prompt yielded both a correct and an incorrect response, and optimise the DPO objective [15] with β=0.1. The reference policy is the base model with the adapter disabled, so no second weight copy is needed.

Self-consistency (SC). No training; at inference we sample K responses and take the majority answer [20].

3.3 Budget accounting

We define

$$\text{GPU-hours} = \frac{t_{\text{wall}} \cdot n_{\text{GPU}}}{3600}, \tag{1}$$

measured around synchronised device work. Sampling and training are *charged*; evaluation is not, because it is identical across methods and independent of the method's cost. Data preparation is not charged. This convention is what makes SC non-free: its entire cost is sampling, and charging it is the point of the comparison. For each budget point we calibrate throughput over a short warm-up and then convert the budget into a fixed number of optimiser steps, so every method at budget b consumes the same b GPU-hours rather than converging independently.

3.4 Evaluation

We report accuracy under three parsers: *lenient* (last number in the output), *standard* (`\boxed{}` if present, else the last number; our main metric), and *strict* (`\boxed{}` only, and only if a non-trivial reasoning trace precedes it). We additionally report format compliance (the fraction of outputs from which the parser could extract anything) and mean output length. Each configuration is run with two seeds; we report per-seed values alongside means.

4 Results

4.1 Accuracy as a function of budget

Table 1 reports accuracy at each budget point, and Figure 1 plots accuracy against cumulative GPU-hours. [TODO: insert the number of the crossover point, if any.]

4.2 The format dividend

[TODO: Table 3: lenient vs. standard vs. strict accuracy per method. State how much of each method's gain is extractable-answer rate rather than correctness.]

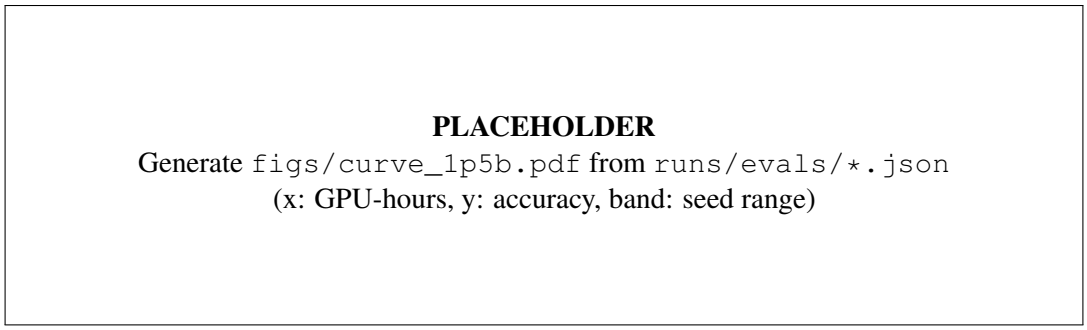


Figure 1: Accuracy vs. cumulative GPU-hours, 1.5B. Shaded band: range across seeds. [TODO: replace with the real figure; regenerate from `runs/evals/*.json`.]

4.3 Where the budget is best spent

[TODO: Interpret the crossover. If SC dominates below some budget b^* and training-side methods dominate above it, report b^* with a confidence interval over seeds. If no crossover exists, say so plainly and report the constant gap.]

5 Threats to Validity

Single scale above 0.5B. Our two model sizes bracket a narrow range. We do not claim the ordering holds at 7B; [19] find RL efficiency saturates with scale, which if anything favours the inference-side baselines at larger sizes, but we have not measured it.

Two seeds. Seed variance is comparable to some of the gaps we report. We therefore report per-seed numbers rather than only means, and we avoid significance claims that two seeds cannot support.

One task family. Mathematical reasoning has a verifiable reward, which favours SC and RFT and disadvantages DPO. We include GSM8K and MATH-500 but not instruction-following, where we expect the ordering to reverse; that is a limitation, not a finding.

Hyperparameters. Matching compute does not match tuning effort. We give every method the same number of hyperparameter trials (reported in the appendix), but a method with a better default inherits an advantage we cannot remove.

6 Conclusion

[TODO: two sentences: what a practitioner with one GPU should do, and what the result does *not* license them to conclude.]

References

[1] Anonymous. Interleaving SFT and RL with buffer. *arXiv preprint arXiv:2510.04454*, 2025.

[2] Anonymous. Large language models post-training: Surveying techniques from alignment to reasoning. *arXiv preprint arXiv:2503.06072*, 2025.

[3] Anonymous. LLM post-training: A deep dive into reasoning large language models. *arXiv preprint arXiv:2502.21321*, 2025.

[4] Anonymous. Quagmires in SFT-RL post-training: When high SFT scores mislead and what to use instead. *arXiv preprint arXiv:2510.01624*, 2025.

[5] Anonymous. SafetyRepro: Configuration-conditional rank instability on alignment benchmarks. *arXiv preprint arXiv:2605.25492*, 2026.

[6] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.

[7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[8] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[9] Hanze Dong, Wei Xiong, Deepanshu Goyal, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023.

[10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[12] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[13] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[15] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[17] Soumye Singhal, Joseph Jennings, Ka Shun Shum, Alexander Bukharin, et al. Priming base models for agentic post-training: An open midtraining recipe. *arXiv preprint arXiv:2610.22019*, 2026.

[18] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.

[19] Zelin Tan, Hejia Geng, Xiaohang Yu, Mulei Zhang, Guancheng Wan, Yifan Zhou, Qiang He, Xiangyuan Xue, Heng Zhou, Yutao Fan, Zhong-Zhi Li, Zaibin Zhang, Guibin Zhang, Chen Zhang, Zhenfei Yin, Philip Torr, and Lei Bai. Scaling behaviors of LLM reinforcement learning post-training: An empirical study in mathematical reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

[20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[22] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A Hyperparameters and full run list

[TODO: auto-generate from `runs/*/meta.json`: every run, its seed, budget, step count, learning rate, wall-clock, and measured GPU-hours. Including the failed runs is what makes Section 3.3 credible.]